AWS Multi-VPC Peering using Terraform

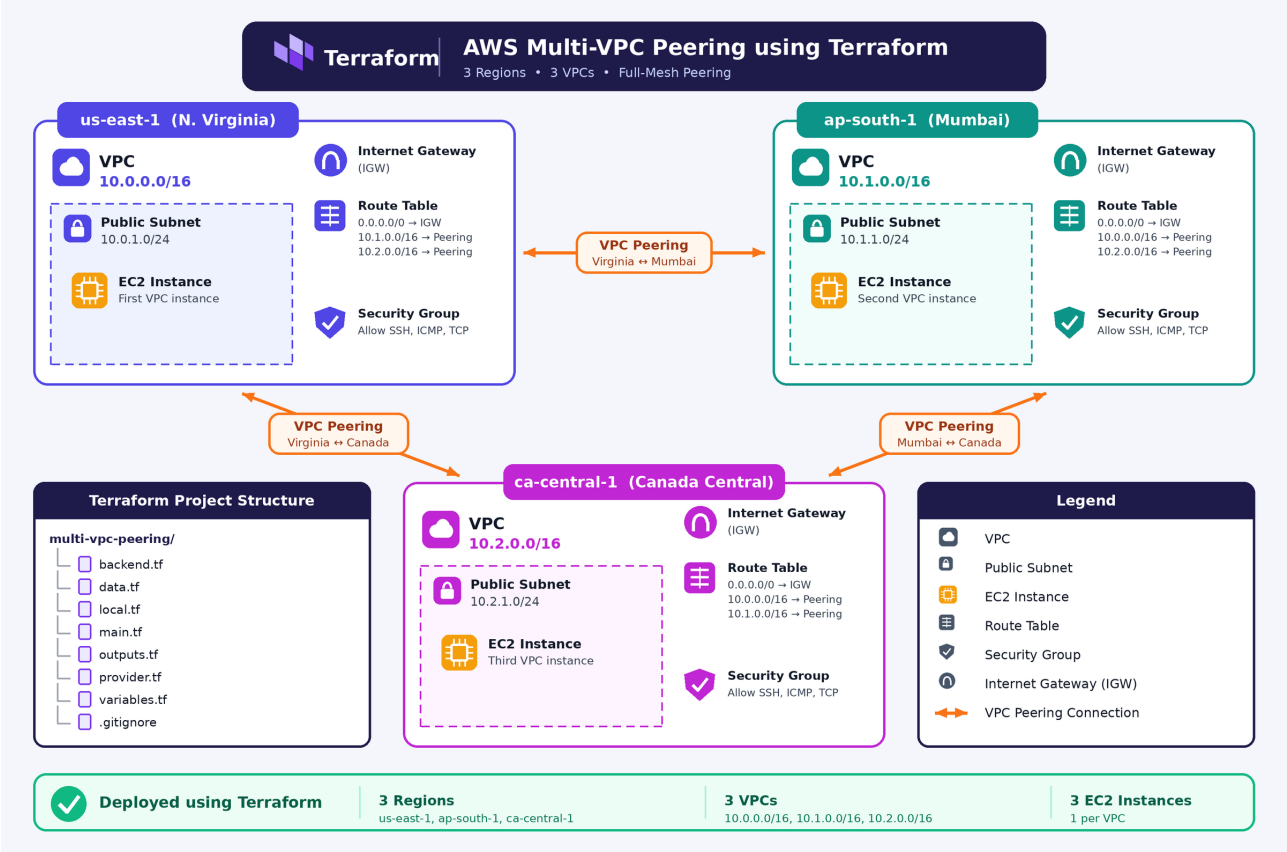
Prepared by: Sakshi Somvanshi

Project Overview

The AWS Multi-VPC Peering using Terraform project demonstrates the implementation of a multi-VPC networking architecture on Amazon Web Services (AWS) using Terraform. In this project, three Virtual Private Clouds (VPCs) are created in three different AWS regions with separate CIDR blocks and connected through VPC Peering connections. Terraform is used to provision and configure the AWS infrastructure, including VPCs, subnets, route tables, Internet Gateways, VPC Peering connections, and EC2 instances.

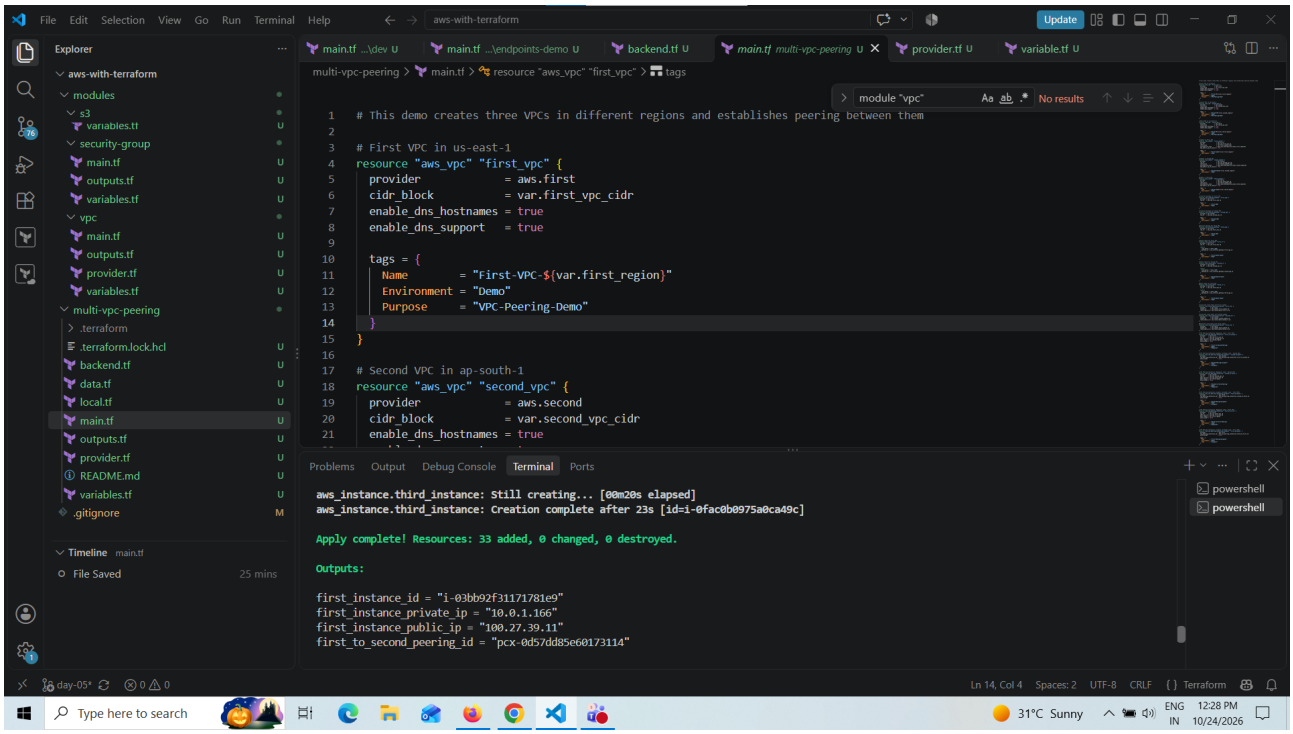
The architecture establishes a full-mesh topology between the three VPCs, allowing private communication between their resources through private IP addresses when the required routes and security rules are configured correctly.

Architecture Diagram



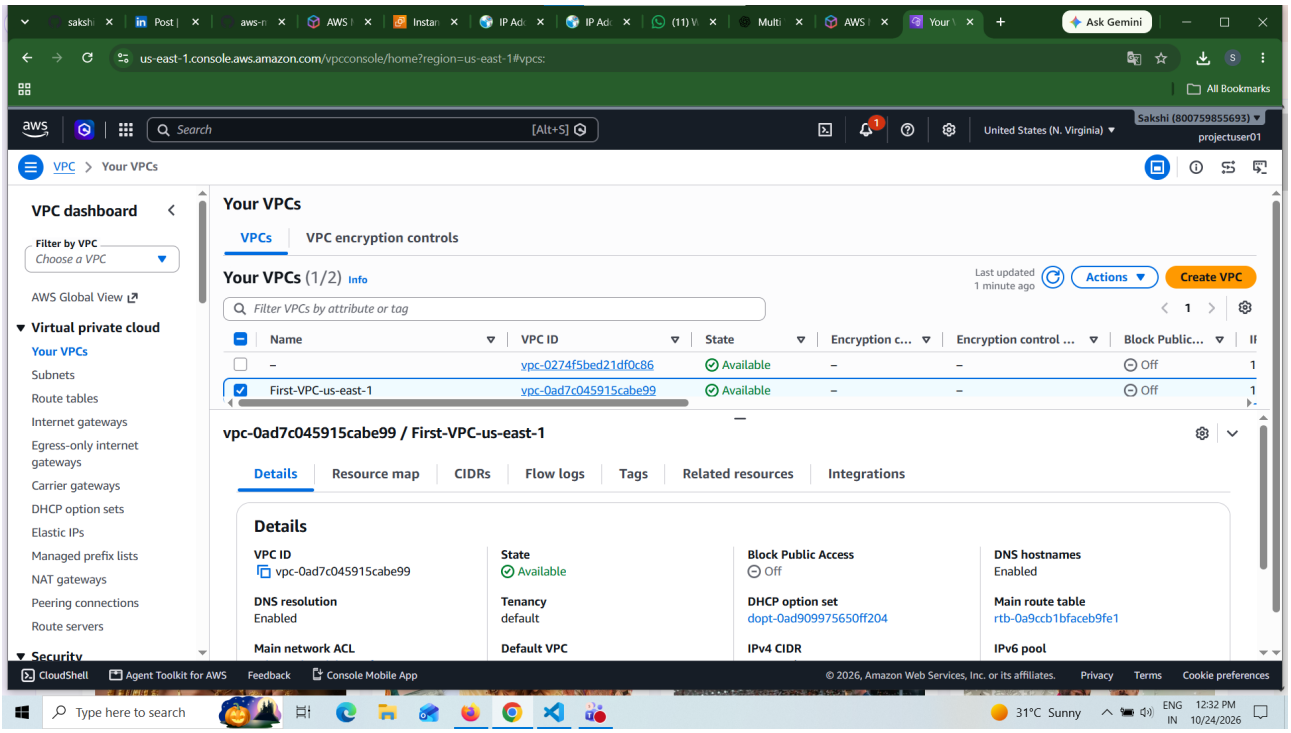
1. Terraform Apply

Executed **terraform apply** to apply the Terraform configuration for the three-VPC architecture. The apply operation completed successfully with **33 resources added, 0 changed and 0 destroyed**, and the outputs (instance IDs, private/public IPs and the first-to-second peering ID) were displayed.

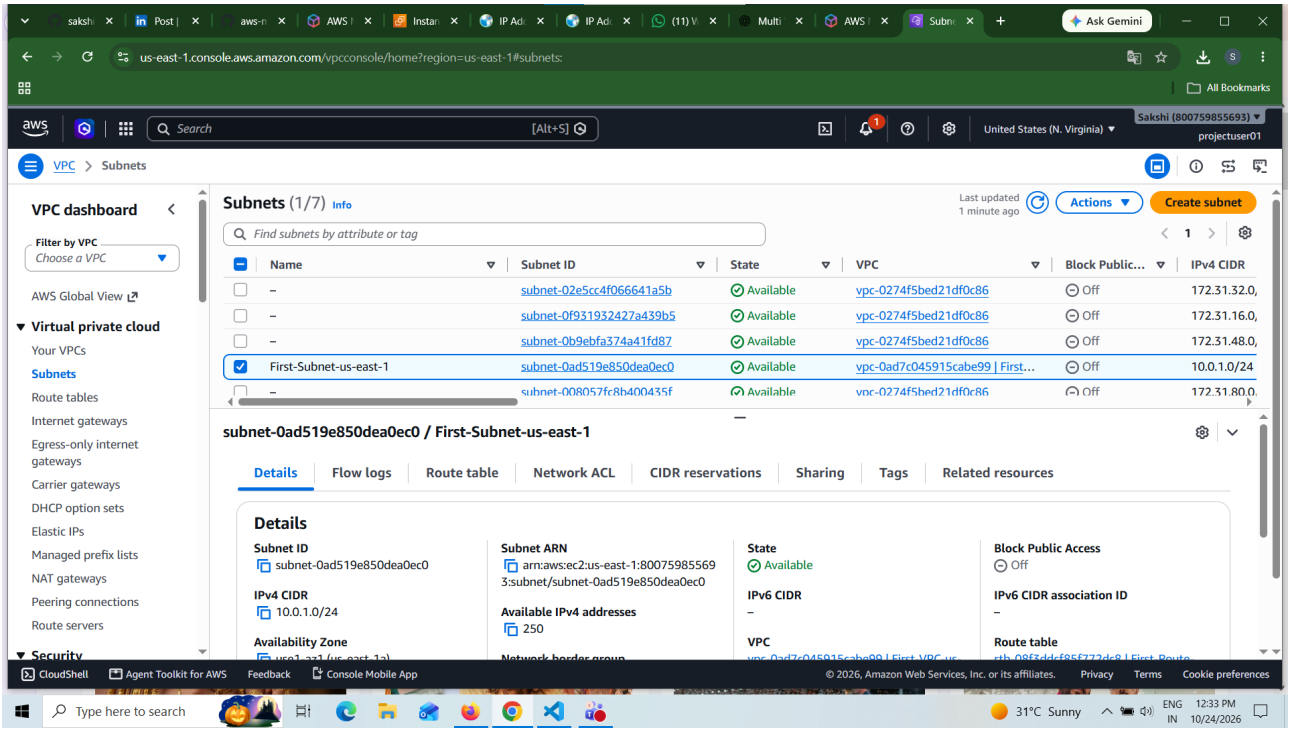


2. Region 1 - us-east-1 (N. Virginia)

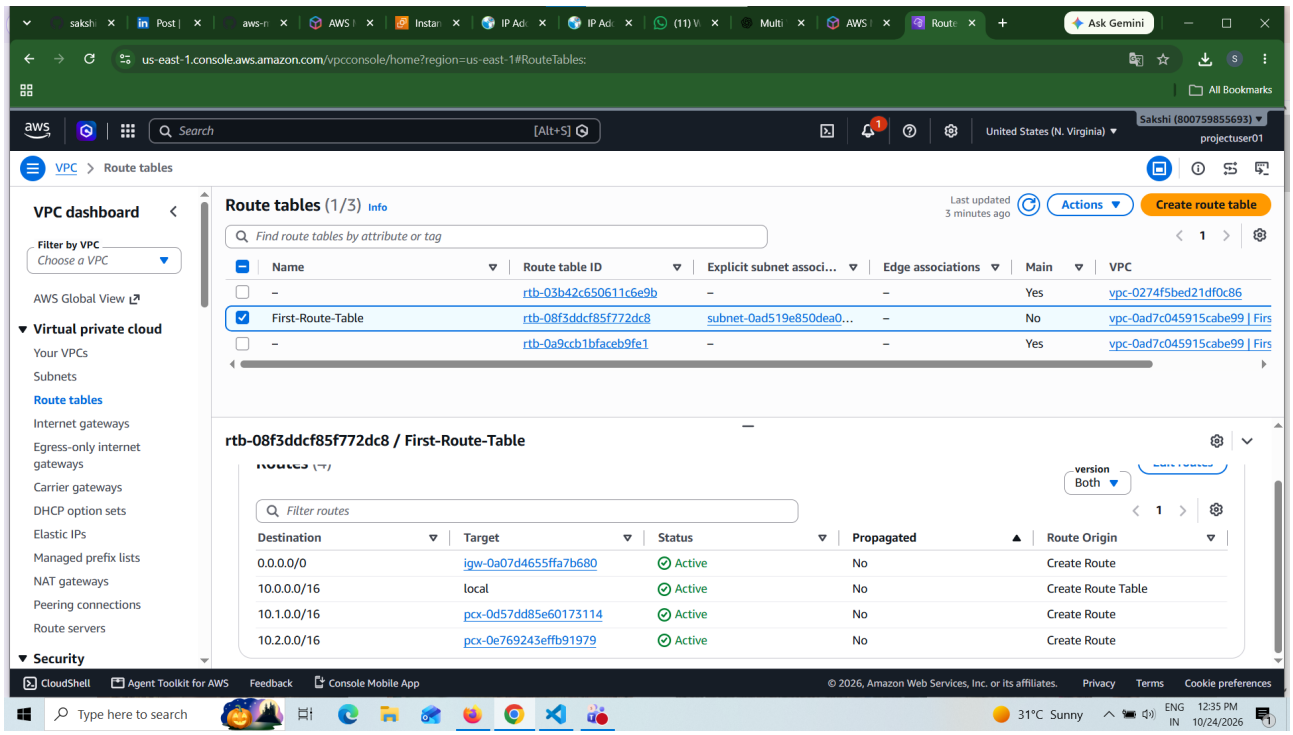
- Created the first VPC (**First-VPC-us-east-1**) in the us-east-1 (N. Virginia) region using Terraform with CIDR block 10.0.0.0/16. The VPC state is Available.



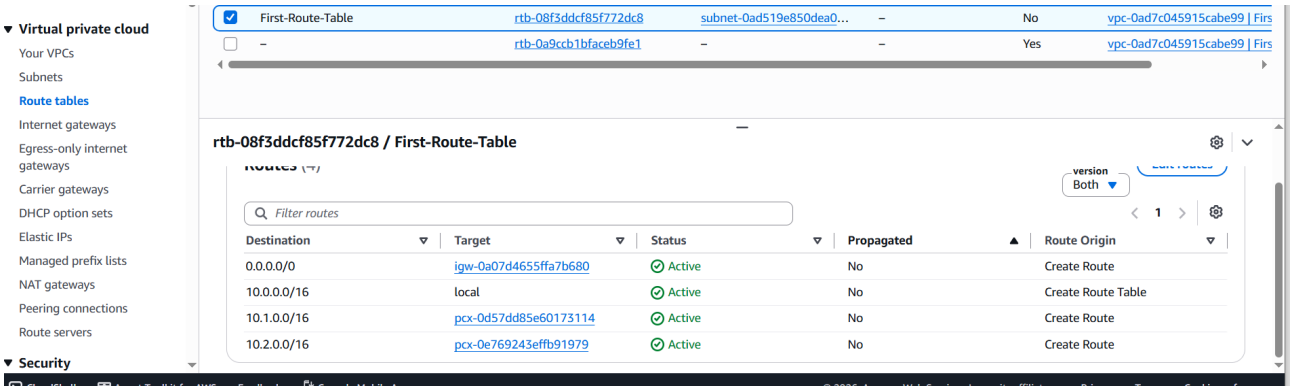
- Created the subnet (**First-Subnet-us-east-1**) in the first VPC using Terraform with CIDR block 10.0.1.0/24.



- Configured the **First-Route-Table** and associated it with the first subnet (10.0.1.0/24). The route table contains the Internet Gateway route (0.0.0.0/0), the local route (10.0.0.0/16) and the VPC Peering routes to 10.1.0.0/16 and 10.2.0.0/16.

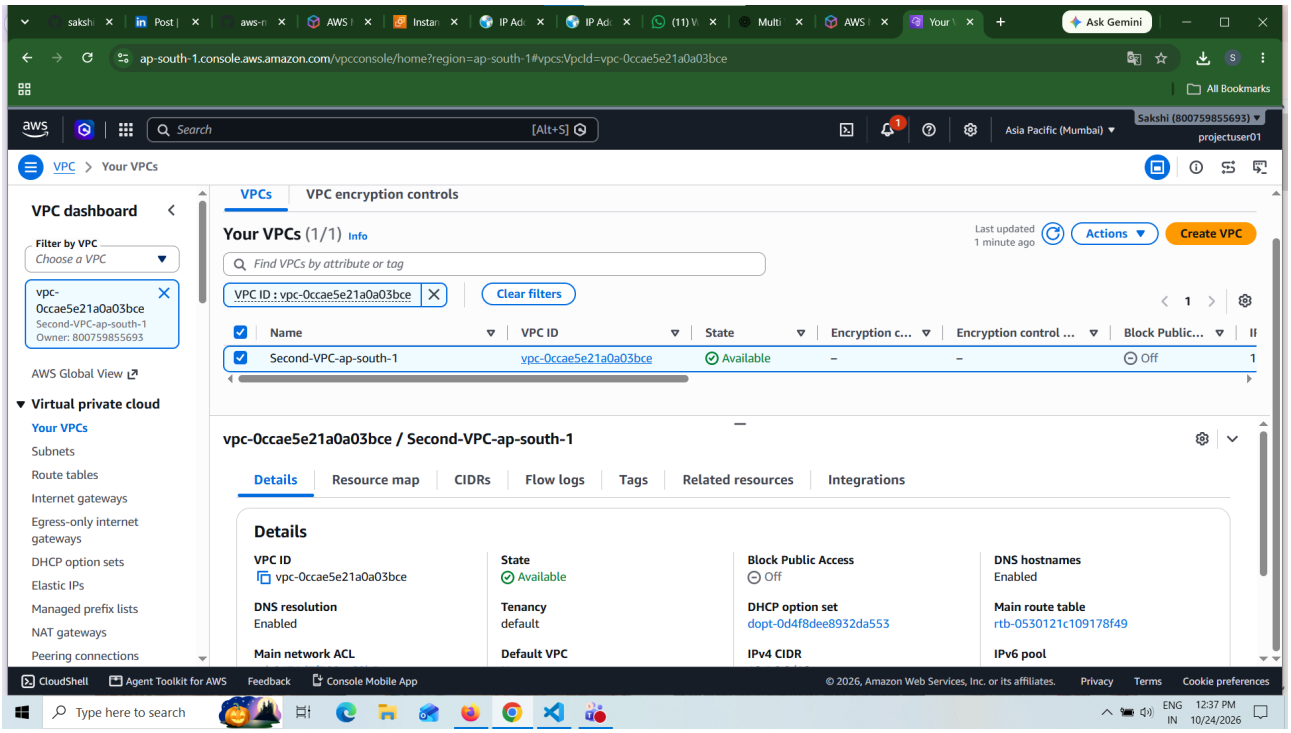


- Route table entries of First-Route-Table: all four routes are in the **Active** state.

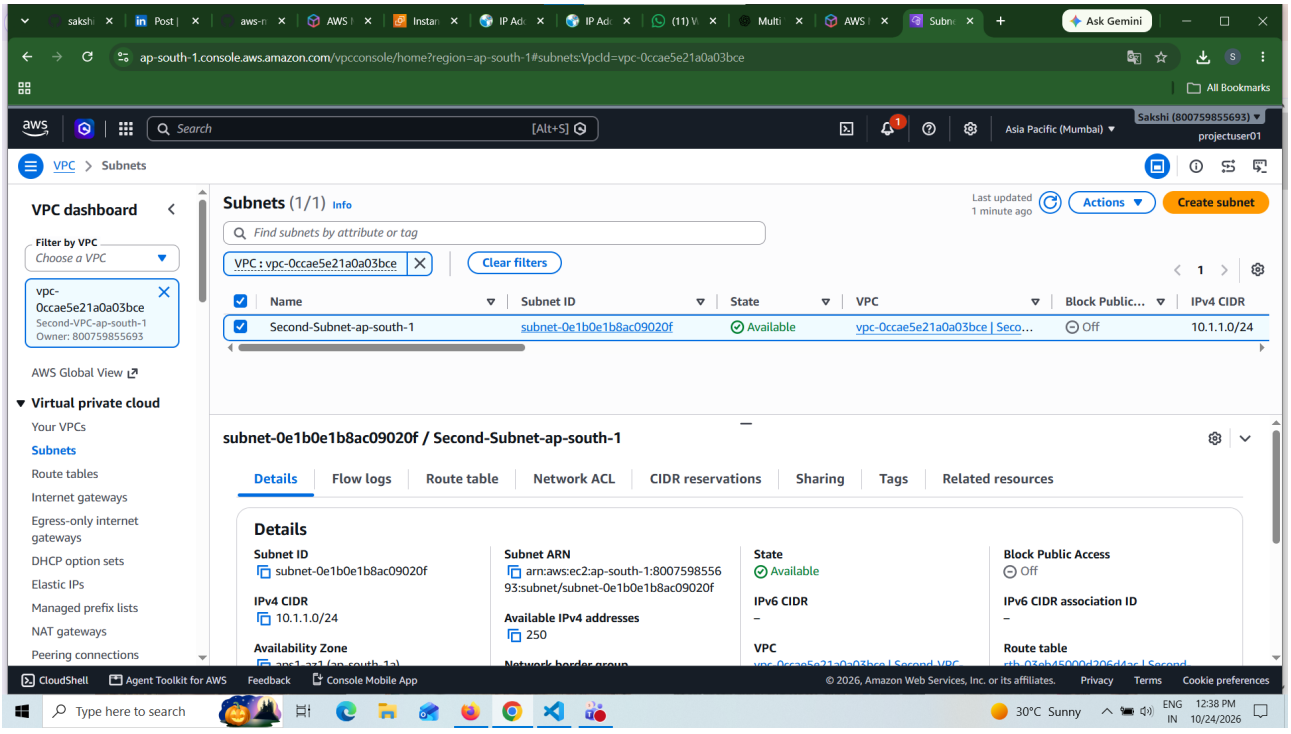


3. Region 2 - ap-south-1 (Mumbai)

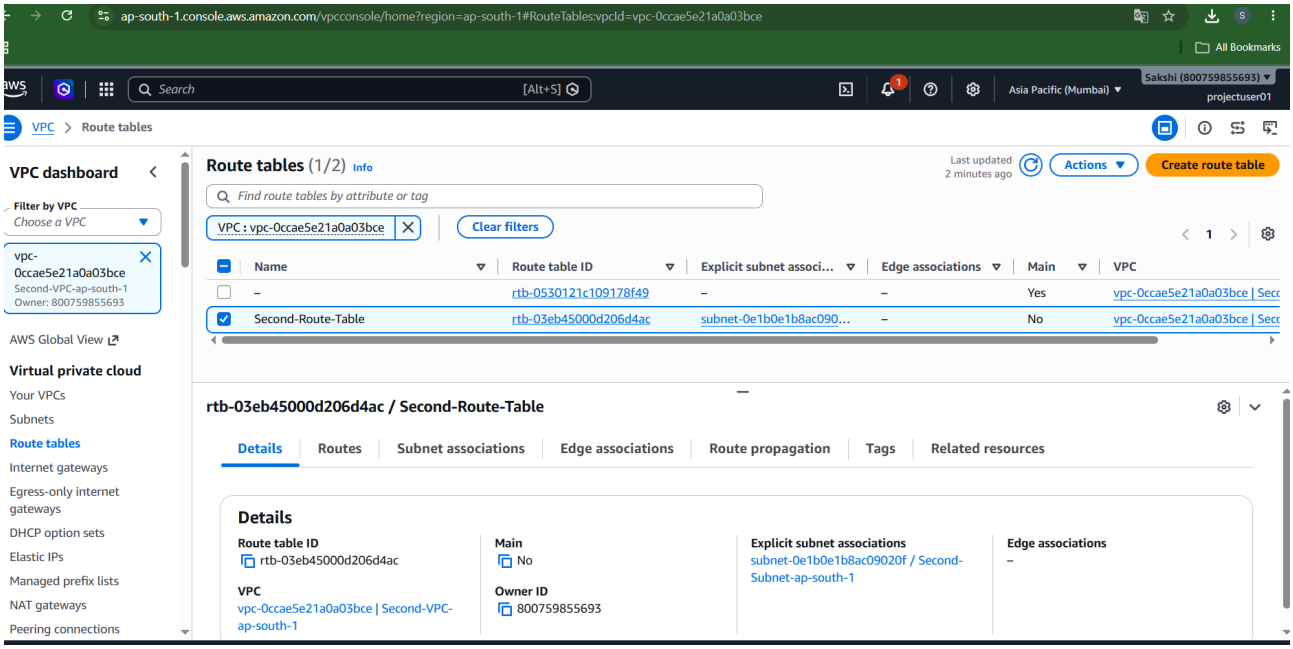
- Created the second VPC (**Second-VPC-ap-south-1**) in the ap-south-1 (Mumbai) region using Terraform with CIDR block 10.1.0.0/16. The VPC state is Available.



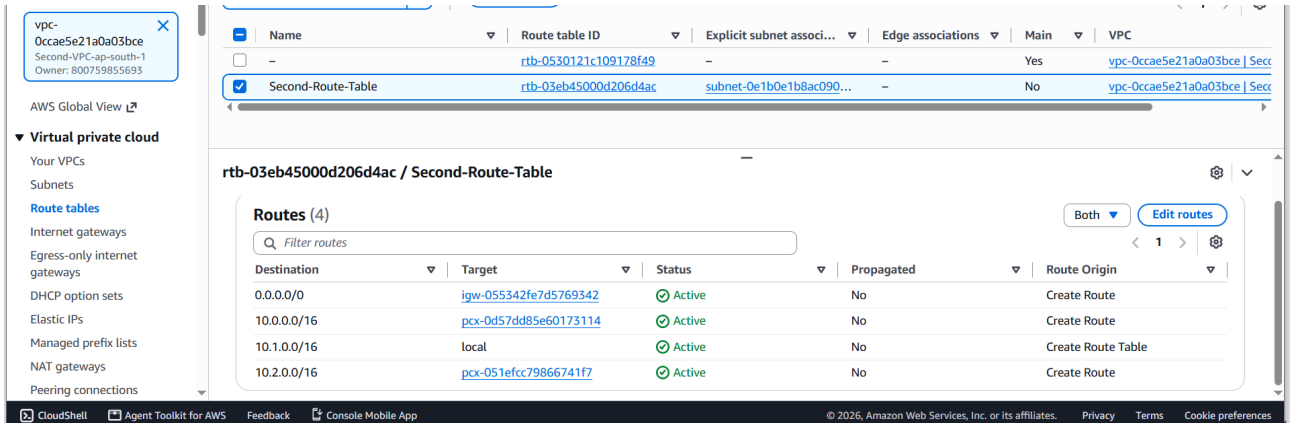
- Created the subnet (**Second-Subnet-ap-south-1**) in the second VPC using Terraform with CIDR block 10.1.1.0/24.



- Configured the **Second-Route-Table** and associated it with the second subnet (10.1.1.0/24) in the Second VPC.

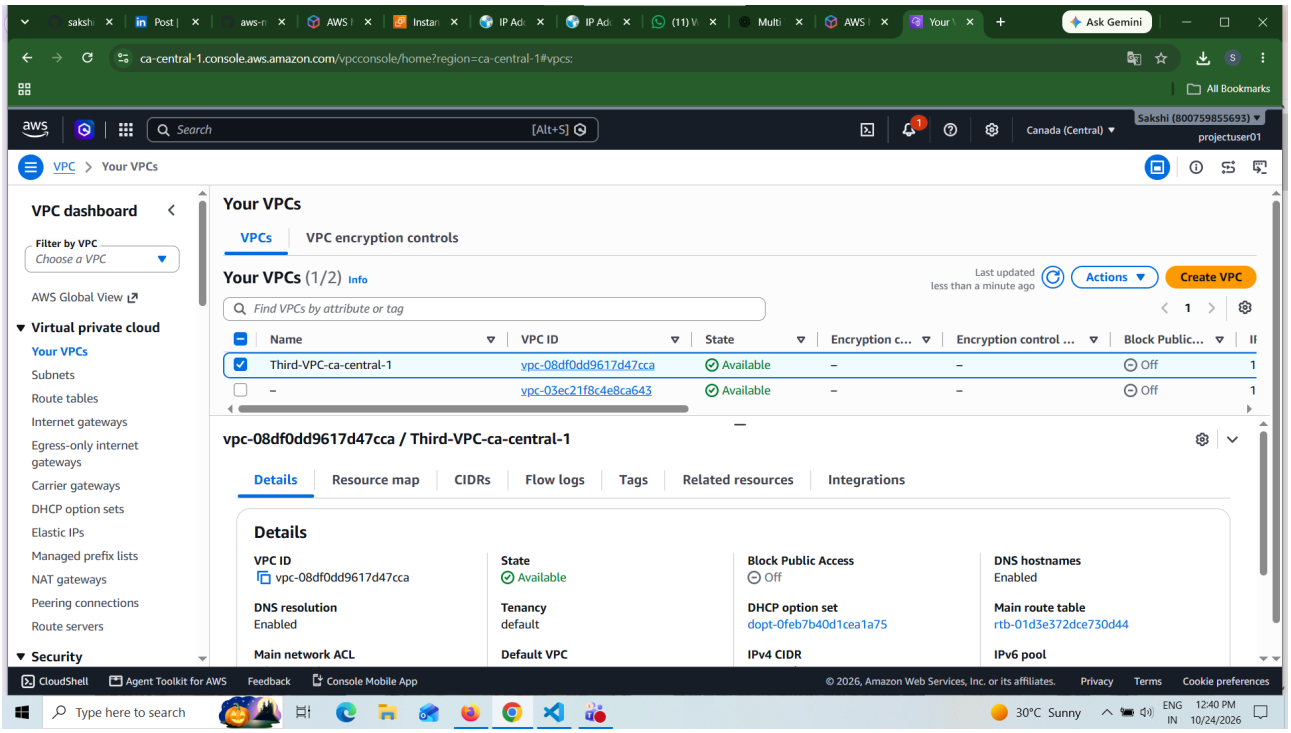


- Route table entries of Second-Route-Table: Internet Gateway route (0.0.0.0/0), VPC Peering routes to 10.0.0.0/16 and 10.2.0.0/16, and the local route (10.1.0.0/16), all **Active**.

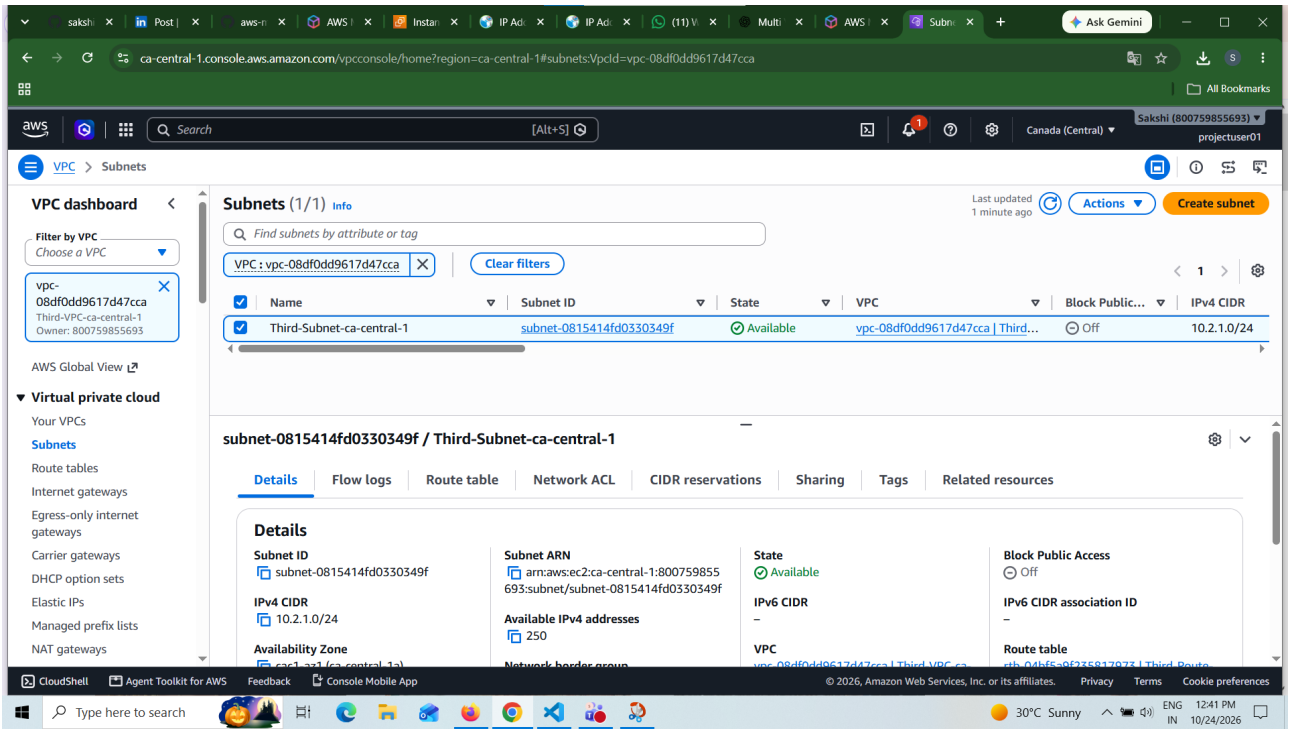


4. Region 3 - ca-central-1 (Canada Central)

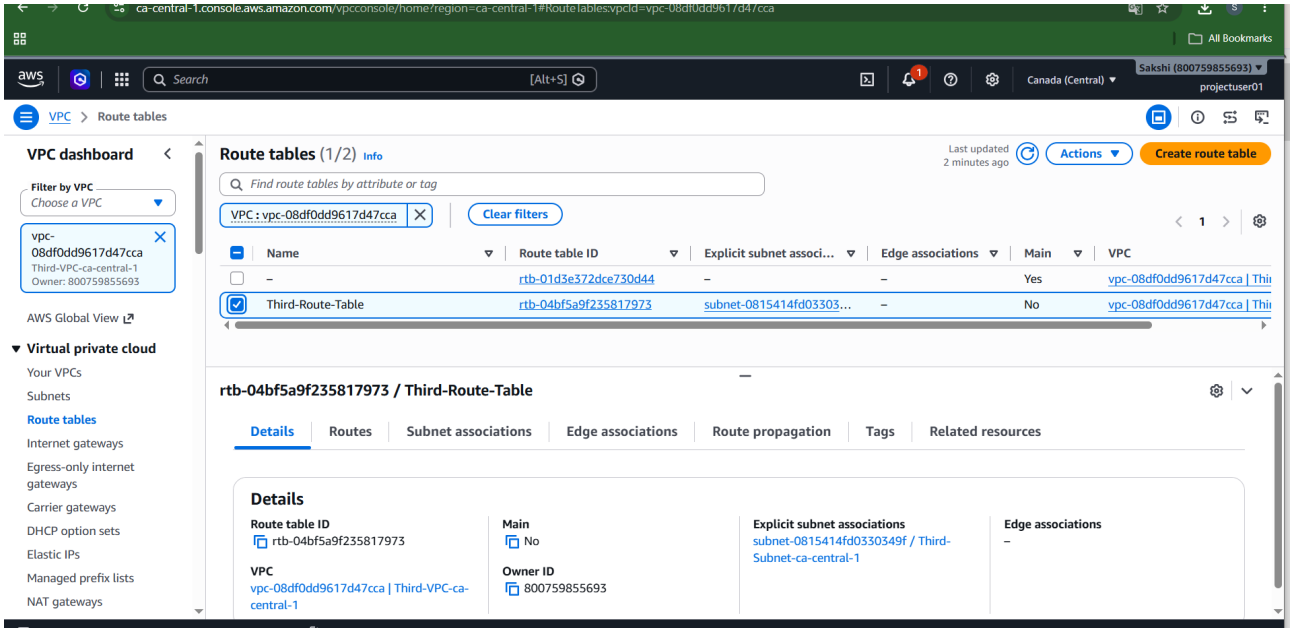
- Created the third VPC (**Third-VPC-ca-central-1**) in the ca-central-1 (Canada Central) region using Terraform with CIDR block 10.2.0.0/16. The VPC state is Available.



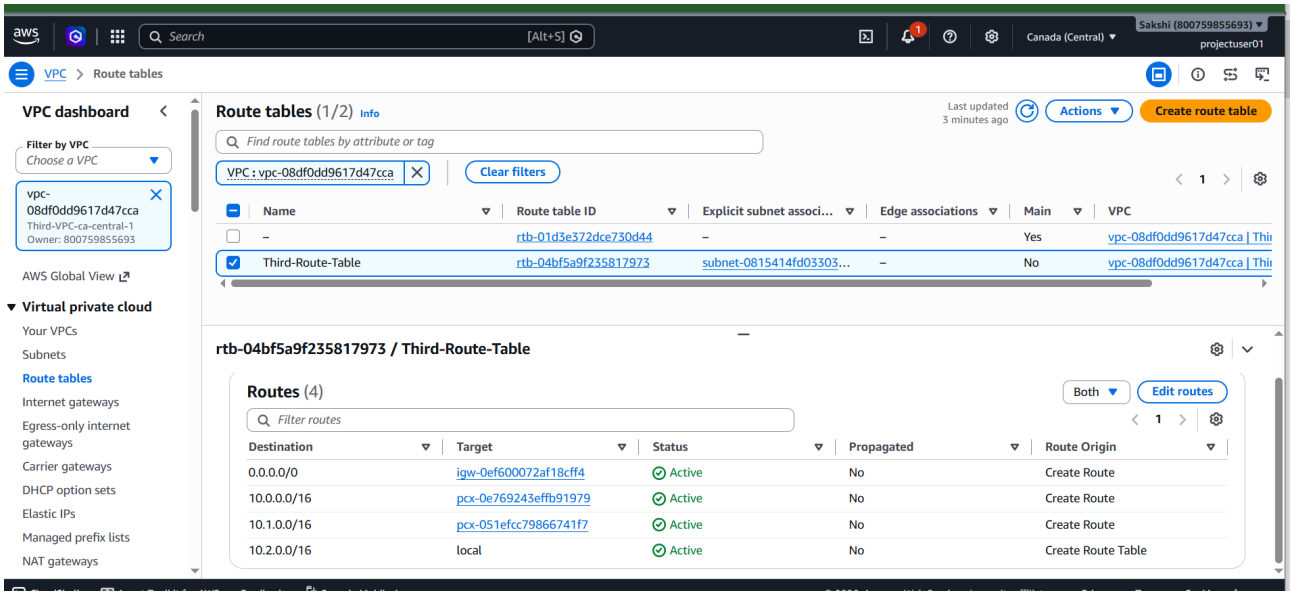
- Created the subnet (**Third-Subnet-ca-central-1**) in the third VPC using Terraform with CIDR block 10.2.1.0/24.



- Configured the **Third-Route-Table** and associated it with the third subnet (10.2.1.0/24) in the Third VPC.



- Route table entries of Third-Route-Table: Internet Gateway route (0.0.0.0/0), VPC Peering routes to 10.0.0.0/16 and 10.1.0.0/16, and the local route (10.2.0.0/16), all **Active**.



Conclusion

Successfully implemented **AWS Multi-VPC Peering using Terraform** by creating three VPCs across different AWS regions (us-east-1, ap-south-1 and ca-central-1) with CIDR blocks 10.0.0.0/16, 10.1.0.0/16 and 10.2.0.0/16. Configured subnets and route tables with peering routes to the other two VPCs using Terraform, establishing a full-mesh VPC Peering topology between the VPCs.